

Assignment Brief

What You Are Building

A RESTful Task Manager API with user authentication. Users must be able to register, log in, and manage their own tasks — create, view, update, and delete them. No user should be able to see or touch another user's data.

Secure Auth	Registration and login with hashed passwords and token-based sessions.
Task CRUD	Full create / read / update / delete for tasks, scoped per user.
Data Persistence	All data stored in a real relational database — nothing in memory.
Clean Structure	Code organised into clear layers: routes, middleware, data access.

Required Tools & Technologies

You must use all of the following. No substitutions.

Tool	Purpose	Why it matters
TypeScript	Language	Type safety — you will rely on it for your data models
Express	HTTP framework	Handles routing, middleware, and request/response lifecycle
Prisma	ORM	Defines your schema and gives you a type-safe database client
PostgreSQL	Database	Relational storage — your schema must include proper relations
JWT	Auth tokens	Stateless session management after login
bcryptjs	Password hashing	Passwords must never be stored in plain text

Features to Implement

User Accounts

- A user can register with their name, email, and a password.
- A user can log in and receive a token to use for subsequent requests.
- Passwords must be hashed before storage — never stored plain.

Task Management

- A logged-in user can create a task with a title, optional description, status, priority, and due date.
- A user can list all their tasks, optionally filtered by status or priority.
- A user can view, update, or delete any individual task they own.
- A user cannot access tasks that belong to someone else.

Data Model

- Users and Tasks must be stored in PostgreSQL via Prisma.
- Tasks must have a status field: To Do, In Progress, or Done.
- Tasks must have a priority field: Low, Medium, or High.
- The relationship between User and Task must be enforced at the database level.

What You Must Showcase

Your submission will be assessed on these six areas. Each one must be clearly demonstrated — either through working endpoints or through your code.

1	Authentication works end-to-end	Register a user, log in, and use the returned token to access a protected route.
2	Middleware protects routes	Any request to a task endpoint without a valid token must be rejected.
3	Data is scoped per user	Demonstrate that User A cannot read, update, or delete User B's tasks.
4	Full task lifecycle	Create a task, update its status to In Progress, then to Done, then delete it.
5	Schema is defined in Prisma	Your schema.prisma must define both models, the relation, and the enums.
6	Error cases are handled	Missing fields, wrong credentials, and not-found resources return clear error responses.

Rules & Reminders

→ **Do not copy code from the reference API.**

You are expected to build this yourself. Understanding comes from doing.

→ **Use TypeScript throughout.**

JavaScript files will not be accepted. Type your models, your requests, your responses.

→ **Your database must be real.**

No in-memory arrays. Prisma must connect to a running PostgreSQL instance.

→ **Structure your project clearly.**

Routes, middleware, and database logic should live in separate files.

→ **Test your own endpoints.**

Use a tool like Postman or cURL to verify every route before submitting.